ФФПМИ

Пайплайн сборки Python пакета

КУРС: ТЕХНОЛОГИИ ПРОГРАММИРОВАНИЯ

Семинар 6.

17/03/2026

Что нужно для корректного запуска файла .py?

Что нужно для корректного запуска файла .py?

1. Интерпретатор Python (какая версия?)
2. Все скачанные библиотеки/модули

Могут ли быть на одном устройстве (компьютере, сервере) разные проекты, которые требуют разные версии интерпретаторов/библиотек?

Что нужно для корректного запуска файла .py?

1. Интерпретатор Python (какая версия?)
2. Все скачанные библиотеки/модули

Могут ли быть на одном устройстве (компьютере, сервере) разные проекты, которые требуют разные версии интерпретаторов/библиотек?

ДА!

Какой есть основной способ сохранить изолированно версии проектов?

Что нужно для корректного запуска файла .py?

1. Интерпретатор Python (какая версия?)
2. Все скачанные библиотеки/модули

Могут ли быть на одном устройстве (компьютере, сервере) разные проекты, которые требуют разные версии интерпретаторов/библиотек?

ДА!

Какой есть основной способ сохранить изолированно версии проектов?

Виртуальное окружение (venv) - папка внутри нашего проекта, которая хранит конкретную версию интерпретатора + все необходимые нам модули/пакеты.

Кто скачивает модули/пакеты (или удаляет их)?

Что нужно для корректного запуска файла .py?

1. Интерпретатор Python (какая версия?)
2. Все скачанные библиотеки/модули

Могут ли быть на одном устройстве (компьютере, сервере) разные проекты, которые требуют разные версии интерпретаторов/библиотек?

ДА!

Какой есть основной способ сохранить изолированно версии проектов?

Виртуальное окружение (venv) - папка внутри нашего проекта, которая хранит конкретную версию интерпретатора + все необходимые нам модули/пакеты.

Кто скачивает модули/пакеты (или удаляет их)?

Пакетный менеджер - pip / uv

Пакетные менеджеры

Создание виртуального окружения:	python -m venv venv (python3.13 -m venv venv)
Вход в виртуальное окружение:	source venv/bin/activate
Выход из виртуального окружения:	deactivate

Начиная с 3.12, pip/uv не дают скачивать модули/пакеты в окружающую среду

PEP668 - https://peps.python.org/pep-0668/

pip

```
pip install numpy
pip uninstall numpy
pip freeze > requirements.txt
pip install -r requirements.txt
```

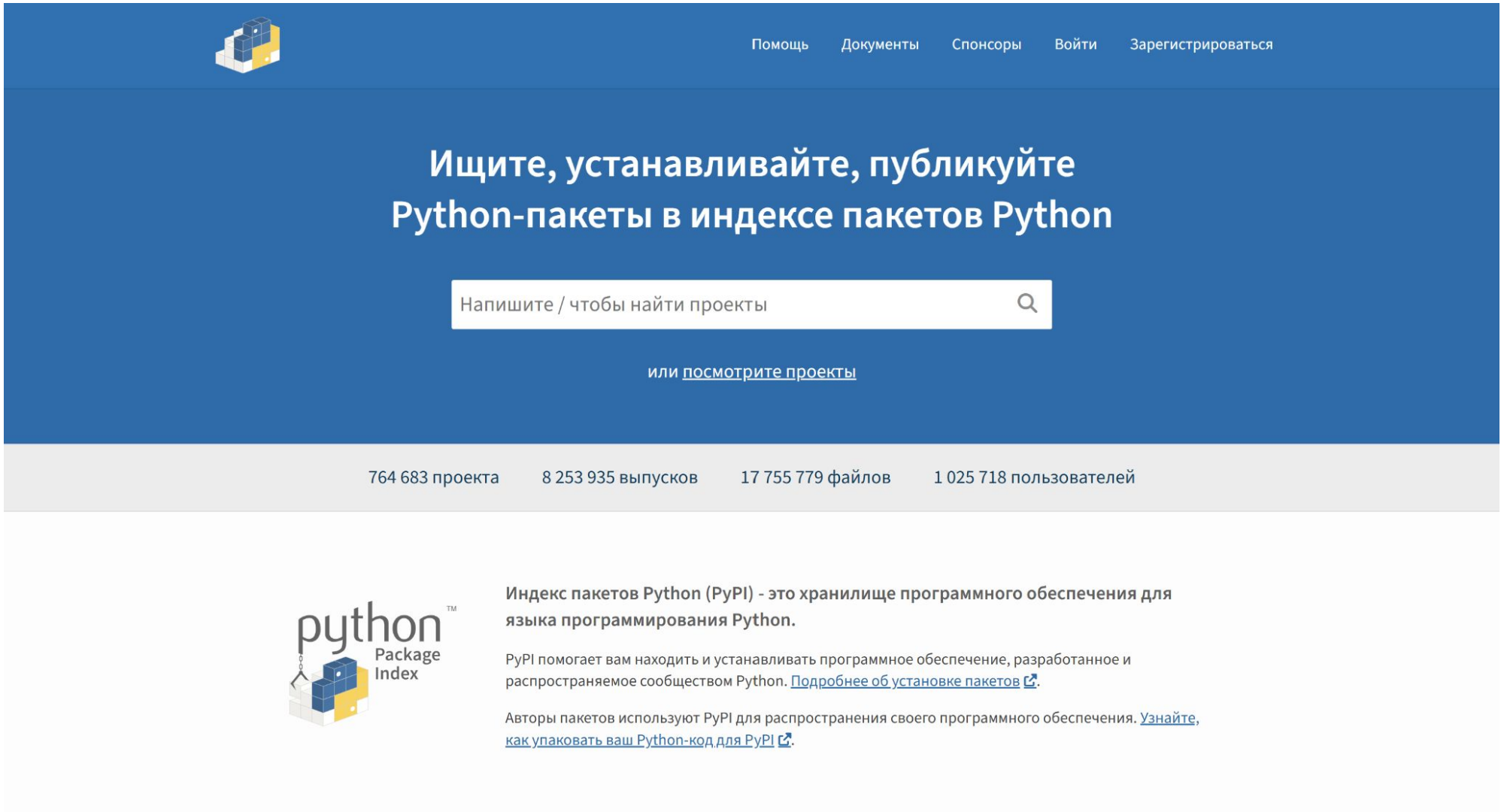
uv

```
uv init my_project
uv add numpy
uv run main.py
```

1) uv быстрее, чем pip в 10-100 раз, так как написан на Rust и активно параллелизует процесс загрузки;
2) uv нужно скачать отдельно, чтобы использовать;
3) uv автоматически работает с виртуальным окружением, не нужно вручную входить/выходить из него;
4) uv поддерживает обратную совместимость - можно писать uv pip install numpy

Где живут пакеты Python?

На PyPi.org - индексе пакетов питона!



Сборка на Python

Вопрос: нужно ли нам ещё что-то в рамках Python, если в целом пакетный менеджер + виртуальное окружение + import за нас всё делает?

Циклические импорты решаются просто хорошей архитектурой решения и import'ами, лежащими под условием (условные импорты - python 3.15)

Да! А что, если мы сами захотим написать свой модуль/пакет?

Модуль - 1 файл с расширением .py;

Пакет - несколько различных файлов в одной директории, формирующих единый блок используемого в Python кода;

Пример большого пакета - https://github.com/scikit-learn/scikit-learn/tree/main

И модуль, и пакет можно импортировать при помощи import. При наличии файла __init__.py в корне пакета, запусится именно он (хотя он не является требованием при формировании пакета начиная с Python 3.3, его наличие всё ещё считается хорошим тоном, так как создаёт единую точку входа другим разработчикам)

__init__.py

Структура проекта:

```
my_project/
    my_package/
        __init__.py
        database.py
        user.py
```

__init__.py

```
print('Инициализация пакета my_package')

from .database import connect
from .user_management import create_user

__all__ = ['connect', 'create_user']  # определяет, что будет доступно при from my_package import *
__author__ = “Nick”
__version__ = “1.2.0”
```

CLI

Мы также можем создавать у пакета интерфейс командной строки (command line interface, CLI). Для этого нужно создать файл __main__.py.
Вызов: команда python -m textutils example.txt

__main__.py

```
import argparse
import sys

def main():
    parser = argparse.ArgumentParser(description="Text utilities CLI")
    parser.add_argument("file", help="Path to text file")
    # тут могут быть другие аргументы для парсинга
    args = parser.parse_args()
    try:
        with open(args.file, "r", encoding="utf-8") as f:
        text = f.read()
    except FileNotFoundError:
        print(f"File not found: {args.file}")
        sys.exit(1)


if __name__ == "__main__":
    main()
```

Зависимости

Если наш пакет использует numpy/pandas определённой версии, можем ли мы просто сохранить весь numpy/pandas и дальше его так отправлять, вместе с другими пакетами?

Зависимости

Если наш пакет использует numpy/pandas определённой версии, можем ли мы просто сохранить весь numpy/pandas и дальше его так отправлять, вместе с другими пакетами?
НЕТ! Тогда мы будем копировать много кода, наши пакеты и проекты будут занимать слишком много памяти + замучаемся с поддерживанием версий :(

__init__.py нужен только интерпретатору. Нам ещё необходим файл, который будет отвечать за именно сборку пакета.

pyproject.toml

```
[project]
name = "cool-ml-lib"
version = "0.1.0"
description = "Example ML library"
authors = [{name = "nick"}]

requires-python = '>=3.12'
dependencies = ["numpy>=1.19.2", "pandas" ]

[build-system]
 requires = ['setuptools>=42', 'wheel']
build-backend = "setuptools.build_meta"
```

Файл **pyproject.toml** нужен для тех, кто будет работать с пакетом извне - пакетные менеджеры и сборщики (pip, poetry, uv, setuptools и.т.д.). Этот файл НЕ импортируется интерпретатором Python.

wheel - это формат пакетов для Python. Пакеты .whl легче и быстрее устанавливать, так как они не требуют компиляции при установке. В отличие от исходных дистрибутивов, которые содержат исходный код и требуют компиляции на этапе установки, пакеты Wheel уже собраны и готовы к использованию. Это особенно важно для пакетов, которые содержат бинарные расширения (например, написанные на C).

Шаги для сборки (при наличии pyproject.toml)

1. pip install build
2. python -m build

Вторая команда создаст два типа пакетов в директории dist:

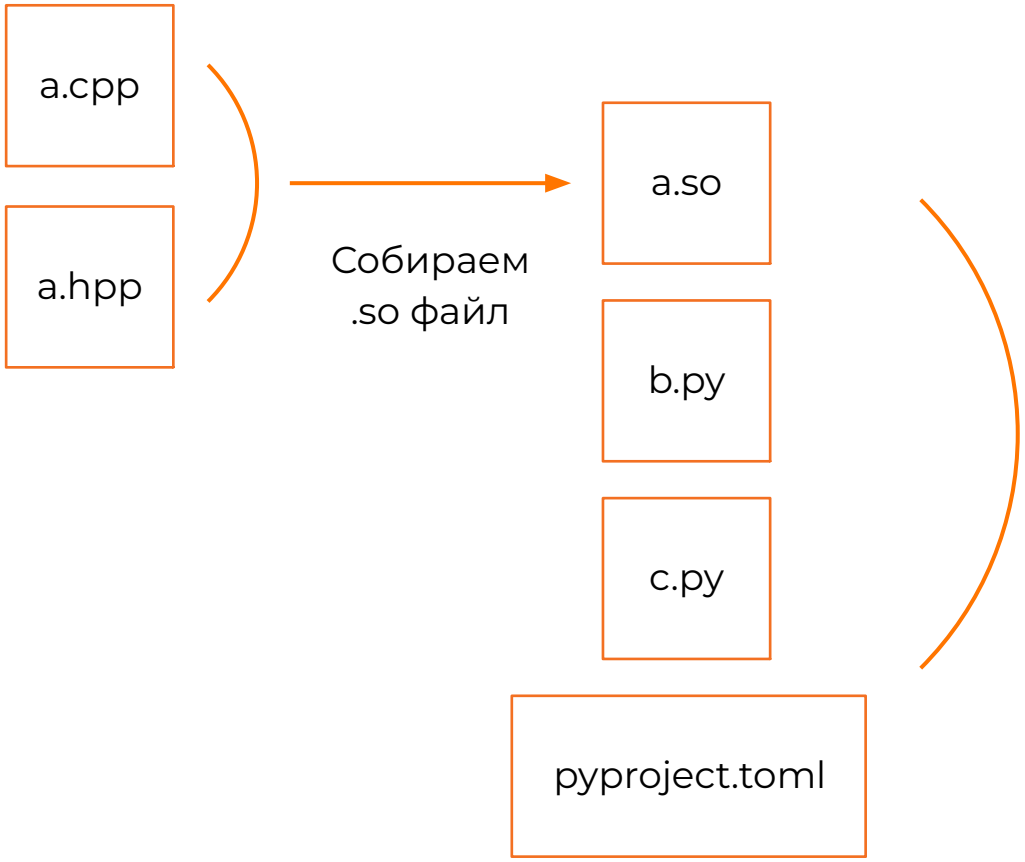
- Source Distribution (sdist) — это архив с исходными файлами проекта (например, .tar.gz).

- Wheel Distribution (wheel) — это собранный и готовый к установке пакет (.whl).

Дальше можно публиковать модуль на PyPI.org через twine.

А какой тогда пайплайн работы в случае, если у нас есть код, скажем, на C++?

Сборка модуля с кодом на C++



so файлы уже можно импортировать в наш Python код при помощи import'ов

Можно и напрямую собирать пакет при помощи pyproject.toml и каких-то сборщиков (в т.ч. meson), но так мы не сможем протестировать код перед сборкой.

Этапы сборки кода на C++

1. Препроцессинг
   Работаем с кодом - как с текстом. Подставляем include'ы и define'ы, обрабатываются директивы.
2. Компиляция
   Переводим наш код в ассемблерный язык
3. Ассемблирование
   Перевод файла на ассемблере в объектный файл (расширение .o)
4. Линковка
   Объединение (связывание) различных объектных файлов.

Вопрос: сможем ли мы всё это сделать при помощи bash, g++, каких-то флагов и скриптов?

Конечно, да. Но это будет неприятно...

Способы сборки

bash

Язык оболочки

setup.sh файл (или любое другое название)

Плюсы:

Мы уже его знаем!

Минусы:

Писать код на баш.....

Нет кроссплатформенности

Нет гибкости (императивный)

https://github.com/akhtyamovpavel/BuildExamples-TP/tree/master/BashExamples

Make

Утилита для автоматизации сборки

Makefile

Плюсы:

Это bash + система таргетов (шаг в сторону декларативности)

Минусы:

Всё ещё почти баш....

Нет кроссплатформенности

https://github.com/akhtyamovpavel/BuildExamples-TP/tree/master/MakeExamples

CMake

Система управления, генерации и автоматизации сборки программного обеспечения

CMakeLists.txt

Плюсы:

Декларативный подход!

Кроссплатформенность! (За счёт генерации скрипта сборки)

Минусы:

(Опционально) сам по себе язык по синтаксису тяжело разобрать/дебажить

https://github.com/akhtyamovpavel/BuildExamples-TP/tree/master/CMakeExamples